

XML - структуриране, валидация, обработка и представяне на документно съдържание

Ивайло Андреев + chat gpt 5.4, 5.5

9 май 2026

Съдържание

1	Добре-структуриран XML	2
1.1	Основни концепции	2
1.2	XML йерархии	2
1.3	Синтактични правила за добре-структуриран XML	2
1.4	Пространства от имена (Namespaces)	3
2	XML валидация чрез DTD	4
2.1	Цели на валидирането	4
2.2	DTD структура и синтаксис	4
3	XML валидация чрез XML Schema	5
3.1	Спецификации и типове данни	6
3.2	Фасети	6
3.3	Структури в XML Schema	6
3.4	Сравнение между DTD и XML Schema	7
4	XSLT и XPath	7
4.1	XSLT	7
4.2	XPath	8
5	Обработка на XML: DOM и SAX	9
5.1	DOM	9
5.2	SAX	9
5.3	Сравнение между DOM и SAX	10
6	Обобщение	10

1 Добре-структуриран XML

XML (*eXtensible Markup Language*) е метаязык за описание на структурирани данни и документи. Основната идея е съдържанието да се представя чрез ясно означени елементи, които са едновременно машинно обработваеми и разбираеми за човека.

1.1 Основни концепции

Дефиниция 1. *Таг е синтактична конструкция от вида `<name>` или `</name>`, с която се отбелязва начало или край на елемент.*

Дефиниция 2. *Елемент е логическа единица в XML документа, съставена от отварящ таг, съдържание и затварящ таг, например `<person>Ivan</person>`.*

Дефиниция 3. *Атрибут е двойка от име и стойност, записана в отварящия таг на елемента, например `id="42"`. Стойността на атрибута винаги се поставя в кавички.*

В началото на документа обикновено стои XML декларация, например:

```
1 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
```

Listing 1: XML declaration example

1.2 XML йерархии

XML документът има дървовидна структура:

- съществува точно един коренов елемент;
- всеки елемент може да има деца, родител и братя;
- елементите образуват логическа и физическа структура на документа.

Логическата структура описва отношенията между елементите, а физическата структура описва от кои единици (entities, външни ресурси) е съставен документът.

1.3 Синтактични правила за добре-структуриран XML

За да бъде един XML документ добре структуриран (*well-formed*), трябва да са изпълнени следните условия:

- всеки отварящ таг има съответен затварящ таг или е самозатварящ се;
- елементите са правилно вложени и не се застъпват;
- имената на елементите са чувствителни към регистъра;

- документът има точно един коренов елемент;
- специалните символи < и & се екранират чрез entities, например < и &;.

Важно е да се различават понятията *well-formed* и *valid*. Един XML документ е **well-formed**, ако спазва синтактичните правила на XML. Един XML документ е **valid**, ако освен това е well-formed и удовлетворява външна формална спецификация, например DTD или XML Schema. Това е класическо разграничение: well-formedness е синтактична коректност, а validity е съответствие със схема.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <catalog>
3   <book id="b1">
4     <title>XML Fundamentals</title>
5     <author>Alice</author>
6   </book>
7   <book id="b2">
8     <title>XPath in Practice</title>
9     <author>Bob</author>
10  </book>
11 </catalog>

```

Listing 2: Well-formed XML example

1.4 Пространства от имена (Namespaces)

Дефиниция 4. *XML пространство от имена* е механизъм за избягване на конфликти между елементи и атрибути с еднакви имена, но различен смисъл.

То се задава чрез атрибута `xmlns`. Често се използва префикс, който се асоциира с URI:

```

1 <root xmlns:h="http://example.org/html"
2   xmlns:f="http://example.org/furniture">
3   <h:table>
4     <h:tr><h:td>Cell</h:td></h:tr>
5   </h:table>
6   <f:table>
7     <f:name>Desk</f:name>
8   </f:table>
9 </root>

```

Listing 3: Namespace example

Префиксите като `h` и `f` са само локални имена в документа. Реалната идентичност на пространството от имена идва от URI. URI не е задължително да сочи към реален ресурс в интернет; той играе ролята на уникален идентификатор.

2 XML валидация чрез DTD

DTD (*Document Type Definition*) описва допустимата структура на XML документ: какви елементи могат да се срещат, в какъв ред, с какви атрибути и с каква краткост.

2.1 Цели на валидирането

Валидацията има следните цели:

- проверка дали документът следва предварително зададена структура;
- осигуряване на съвместим обмен на данни между системи;
- повторна употреба на една и съща структурна дефиниция за много документи.

Добре-структурираният документ е синтактично коректен, а валидният документ е добре-структуриран и допълнително удовлетворява дадена DTD или XML Schema.

2.2 DTD структура и синтаксис

DTD може да бъде:

- **вътрешна** — дефинирана в самия XML документ;
- **външна** — записана в отделен файл и реферирана от XML документа.

Пример за вътрешна DTD декларация:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE catalog [
3   <!ELEMENT catalog (book+)>
4   <!ELEMENT book (title, author)>
5   <!ATTLIST book id ID #REQUIRED>
6 ]>
7 <catalog>
8   <book id="b1">
9     <title>XML</title>
10    <author>Alice</author>
11  </book>
12 </catalog>
```

Listing 4: Internal DTD declaration example

Пример за външна DTD декларация:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE catalog SYSTEM "catalog.dtd">
3 <catalog>
4   <book id="b1">
```

```

5      <title>XML</title>
6      <author>Alice</author>
7    </book>
8 </catalog>

```

Listing 5: External DTD declaration example

Основните DTD декларации са:

- `<!ELEMENT ...>` — описание на допустимото съдържание на елемент;
- `<!ATTLIST ...>` — описание на атрибутите на елемент;
- `<!ENTITY ...>` — описание на entities.

Често използвани модели на съдържание са:

- `EMPTY` — празен елемент;
- `ANY` — произволно съдържание;
- `(#PCDATA)` — текстово съдържание.

Операторите за кратност са:

- `?` — нула или един път;
- `*` — нула или повече пъти;
- `+` — един или повече пъти.

```

1 <!ELEMENT catalog (book+)>
2 <!ELEMENT book (title, author, year?)>
3 <!ATTLIST book id ID #REQUIRED>
4 <!ELEMENT title (#PCDATA)>
5 <!ELEMENT author (#PCDATA)>
6 <!ELEMENT year (#PCDATA)>

```

Listing 6: DTD example

При атрибутите често се използват типовете `CDATA`, `ID`, `IDREF` и ограниченията `#REQUIRED`, `#IMPLIED`, `#FIXED`.

DTD е полезна за базово структурно валидиране, но има съществени ограничения: не поддържа богати типове данни като числа и дати и няма естествена пълна поддръжка за namespaces. Именно това е една от основните причини XML Schema да се използва като по-мощен наследник на DTD.

3 XML валидация чрез XML Schema

XML Schema (XSD) е по-мощен език за описание на XML структури от DTD. За разлика от DTD, самата схема е XML документ и поддържа богата типова система.

3.1 Спецификации и типове данни

XSD предлага:

- примитивни типове като `string`, `boolean`, `decimal`, `date`;
- производни типове като `integer`, `long`, `language`, `IDREF`;
- потребителски типове, създадени чрез ограничаване или разширяване на съществуващи типове.

3.2 Фасети

Дефиниция 5. *Фасети са ограничения върху множеството допустими стойности на даден тип.*

Примери за фасети са:

- `length`, `minLength`, `maxLength`;
- `minInclusive`, `maxInclusive`;
- `pattern`;
- `enumeration`.

Кратък пример: чрез `enumeration` можем да ограничим стойността на атрибут или елемент до краен списък, например `small`, `medium`, `large`. Чрез `pattern` можем да наложим формат, например пощенски код от точно 4 цифри.

3.3 Структури в XML Schema

Важно е да се прави разлика между **simple content** и **complex content**. При `simple content` елементът съдържа само текстова стойност и няма вложени елементи. При `complex content` елементът може да съдържа други елементи и/или атрибути.

Основни конструкции в XSD са:

- `<xs:element>` и `<xs:attribute>` — декларации на елементи и атрибути;
- `<xs:simpleType>` — тип за `simple content`, тоест само текстова стойност;
- `<xs:complexType>` — тип за `complex content`, тоест вложени елементи и/или атрибути;
- `<xs:sequence>` — подредена последователност;
- `<xs:all>` — всички изброени елементи, без задължителен ред;

- `<xs:choice>` — избор на един от няколко възможни елемента.

```

1 <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
2   <xs:element name="book">
3     <xs:complexType>
4       <xs:sequence>
5         <xs:element name="title" type="xs:string"/>
6         <xs:element name="author" type="xs:string"/>
7         <xs:element name="year" type="xs:integer" minOccurs="0"/>
8       </xs:sequence>
9       <xs:attribute name="id" type="xs:ID" use="required"/>
10    </xs:complexType>
11  </xs:element>
12 </xs:schema>

```

Listing 7: XSD example

3.4 Сравнение между DTD и XML Schema

Характеристика	DTD	XML Schema
Синтаксис	Собствен синтаксис	XML синтаксис
Типове данни	Силно ограничени	Богата типова система
Namespaces	Ограничена поддръжка	Пълна поддръжка
Ограничения	По-базови	Фасети и сложни правила

4 XSLT и XPath

Технологиите XSLT и XPath се използват за избор, трансформация и представяне на XML данни.

4.1 XSLT

XSLT (*eXtensible Stylesheet Language Transformations*) е език за трансформиране на XML документи в други формати, например HTML, текст или друг XML документ.

Обработката в XSLT е **template-based** и декларативна: описваме шаблони, които казват какви възли да бъдат разпознати и какъв резултат да се конструира, вместо да пишем императивен алгоритъм стъпка по стъпка.

Основни XSLT конструкции са:

- `<xsl:template>` — шаблон за обработка на възли;
- `<xsl:value-of>` — извличане на стойност;
- `<xsl:for-each>` — обхождане на множество възли;
- `<xsl:apply-templates>` — прилагане на шаблони върху децата на текущия възел.

```

1 <xsl:stylesheet version="1.0"
2   xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
3   <xsl:template match="/catalog">
4     <html>
5       <body>
6         <ul>
7           <xsl:for-each select="book">
8             <li><xsl:value-of select="title"/></li>
9           </xsl:for-each>
10        </ul>
11      </body>
12    </html>
13  </xsl:template>
14 </xsl:stylesheet>

```

Listing 8: XSLT example

XSLT позволява ясно разделяне между данни и представяне: XML пази съдържанието, а XSLT описва как то да бъде визуализирано или преобразувано.

4.2 XPath

Дефиниция 6. ***XPath** е език за адресиране и навигация в XML документ, който разглежда документа като дърво от възли.*

XPath използва синтаксис, подобен на файлови пътища. Основни оператори са:

- / — абсолютен път от корена;
- // — търсене на възли навсякъде в документа;
- @ — избор на атрибут;
- .. — избор на родителски възел.

Особено важни са и **предикатите**, с които се филтрират възли:

- [1] — първият елемент от избраното множество;
- [last()] — последният елемент;
- [price > 30] — елементи, за които вложеният елемент **price** има стойност над 30.

Примери за XPath изрази:

- /catalog/book/title — всички заглавия на книги в корена **catalog**;
- //book[@id='b1'] — книга с атрибут **id='b1'**;
- //author — всички елементи **author** в документа;

- `/catalog/book[1]` — първата книга в каталога;
- `//book[last()]` — последната книга в документа;
- `//book[price > 30]` — всички книги с цена над 30.

5 Обработка на XML: DOM и SAX

5.1 DOM

DOM (*Document Object Model*) е **tree-based API**: представя целия XML документ като дърво в паметта.

Основни интерфейси в DOM са:

- **Node** — базов интерфейс за възел;
- **Document** — кореновият обект на документа;
- **Element** — елементен възел;
- **Attr** — атрибутен възел;
- **Text** — текстов възел.

Предимството на DOM е, че позволява произволен достъп, навигация и модификация на документа. Недостатък е по-високата консумация на памет при големи файлове.

5.2 SAX

SAX (*Simple API for XML*) е **event-driven streaming API**: XML документът се чете последователно, а при срещане на отварящ таг, затварящ таг, текст или грешка парсерът генерира събития.

Основни интерфейси в SAX са:

- **XMLReader**;
- **ContentHandler**;
- **ErrorHandler**.

Приложението обработва тези събития чрез обработчици на събития като **ContentHandler** и **ErrorHandler**. SAX е особено подходящ за много големи документи, защото не съхранява цялото дърво в паметта.

5.3 Сравнение между DOM и SAX

Характеристика	DOM	SAX
Модел	Дърво в паметта	Поток от събития
Памет	Висока консумация	Ниска консумация
Достъп	Произволен	Последователен
Модификация	Удобна	Ограничена
Подходящ за	Сложни промени	Много големи файлове

6 Обобщение

XML е универсален стандарт за структуриране и обмен на документно съдържание. Добре-структурираният XML осигурява синтактична коректност, DTD и XML Schema осигуряват валидиране спрямо формална спецификация, XSLT и XPath позволяват трансформация и навигация, а DOM и SAX дават два различни програмни модела за обработка на документи според нуждите на приложението.